

# Running Time and Asymptotic Notation

Data Structures & Algorithms

---

Grading scheme:

- Homework 20%
- In-class quizzes 20% (2 lowest scores dropped)
- 2 exams 60%

Grading scheme:

- Homework 20%
- In-class quizzes 20% (2 lowest scores dropped)
- 2 exams 60%

You will need a laptop or tablet for in-class quizzes or exams. Do not change browser tabs during quizzes or exams.

Grading scheme:

- Homework 20%
- In-class quizzes 20% (2 lowest scores dropped)
- 2 exams 60%

You will need a laptop or tablet for in-class quizzes or exams. Do not change browser tabs during quizzes or exams.

All announcements, grades, and homework will be posted on Canvas. Check it regularly.

# Generative AI Usage Guidelines

- Asking **clarifying** questions related to lectures, homework, or exams is OK. Treat it as a tutor.
- For homework, you may use AI to do sanity checks on your solutions **without revealing the solution. See the syllabus for an example prompt.**

The homework is only 20% of your grade, so treat it as a learning opportunity.

I do believe that AI is a powerful tool that can help you learn, but it is not a substitute for learning itself.

## My take

We will operate based on the premise that you believe in the value of learning.

- Grow cognitive skills by doing the work yourself
- Understanding is important for both academic and professional success
- There is value in learning in itself

An algorithm is a **precise, unambiguous** set of instructions that solves a problem.

Analyzing one has two parts:

An algorithm is a **precise, unambiguous** set of instructions that solves a problem.

Analyzing one has two parts:

- Prove it is **correct**.
- Evaluate its **efficiency**: time, memory, ...



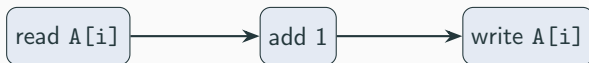
## Running Time

---

# Counting Instructions

Measure running time by the number of **basic machine instructions**: memory accesses, pairwise arithmetic.

$A[i] = A[i] + 1$



Count instructions **up to a constant factor**.

# Selection Sort: The Idea

Scan  $A[1:n]$  for the smallest element, put it in  $A[1]$ . Then scan  $A[2:n]$ , put its smallest in  $A[2]$ . And so on.

$j = 1$ : 

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 2 | 9 | 1 | 7 |
|---|---|---|---|---|

$j = 2$ : 

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 9 | 5 | 7 |
|---|---|---|---|---|

$j = 3$ : 

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 9 | 5 | 7 |
|---|---|---|---|---|

$j = 4$ : 

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 5 | 9 | 7 |
|---|---|---|---|---|

done: 

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 5 | 7 | 9 |
|---|---|---|---|---|

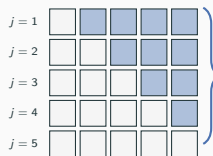
sorted prefix    minimum of the rest

# Selection Sort

```
function selection_sort!(A)
    n = length(A) # ln 1
    for j in 1:n-1 # ln 2
        min_idx = j # ln 3
        for k in j+1:n # ln 4
            if A[min_idx] > A[k] # ln 5
                min_idx = k # ln 6
            end
        end
        A[j], A[min_idx] = A[min_idx], A[j] # ln 7
    end
    return A
end
```

# Counting the Lines

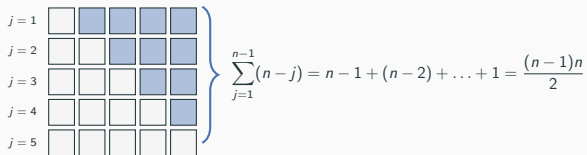
- Lines 1, 2, 3 run  $n - 1$  times.
- Lines 4, 5, 6 run at most  $(n - 1) + (n - 2) + \dots + 1$  times.
- Line 7 runs  $n - 1$  times.


$$\left. \begin{array}{l} j=1 \\ j=2 \\ j=3 \\ j=4 \\ j=5 \end{array} \right\} \sum_{j=1}^{n-1} (n-j) = n-1 + (n-2) + \dots + 1 = \frac{(n-1)n}{2}$$

Recall  $1 + 2 + \dots + n = \frac{n(n+1)}{2}$ .

# Counting the Lines

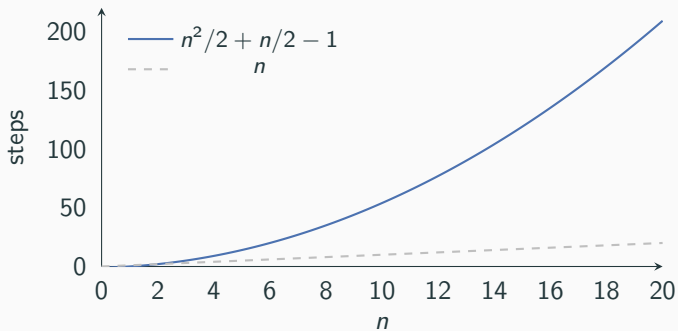
- Lines 1, 2, 3 run  $n - 1$  times.
- Lines 4, 5, 6 run at most  $(n - 1) + (n - 2) + \dots + 1$  times.
- Line 7 runs  $n - 1$  times.



Recall  $1 + 2 + \dots + n = \frac{n(n+1)}{2}$ .

## Total

$$4(n-1) + 3\frac{(n-1)n}{2} + 1 = O(n^2)$$



The running time grows **quadratically** in  $n$ .

# Exercise

What is the running time of this function in terms of  $n$ ?

```
function dummy_algorithm(n)
  for j in 1:n
    x = 0
    for k in 1:2^j
      y = 1
    end
  end
end
```

**Hint: geometric series**

$$\sum_{i=0}^m r^i = \frac{r^{m+1} - 1}{r - 1}, \quad r \neq 1.$$



The inner loop runs  $2^j$  times for each  $j$ . So the total work is

$$\sum_{j=1}^n 2^j = \left( \sum_{j=0}^n 2^j \right) - 1 = \frac{2^{n+1} - 1}{2 - 1} - 1 = 2^{n+1} - 2.$$

## Answer

$O(2^n)$ : the last iteration alone dominates everything before it.

# Asymptotic Notation

---

# Why Asymptotics

For  $T(n) = n^2/2 + n/2 - 1$ , the term  $n^2$  **dictates** the growth.

$$\frac{n^2}{2}$$

dominant

$$\frac{n}{2}$$

noise

$$-1$$

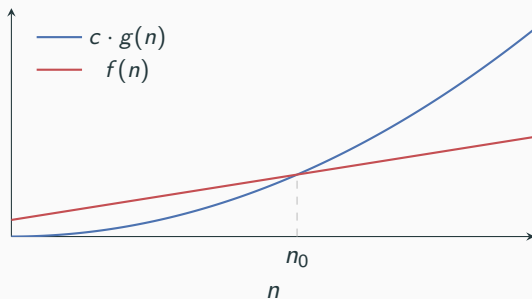
noise

We write  $f(n) = O(g(n))$  to say:  $f$  grows **no more** than  $g$ .

## Definition

$f(n) = O(g(n))$  if there exist constants  $c > 0$  and  $n_0$  such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$



## Example

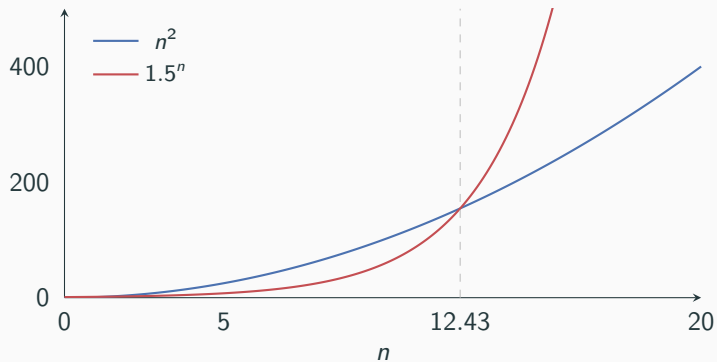
$$3n^2 + 5n + 10 = O(n^2)$$

Take  $c = 18$  and  $n_0 = 1$ . For  $n \geq 1$ :

$$3n^2 + 5n + 10 \leq 3n^2 + 5n^2 + 10n^2 = 18n^2.$$

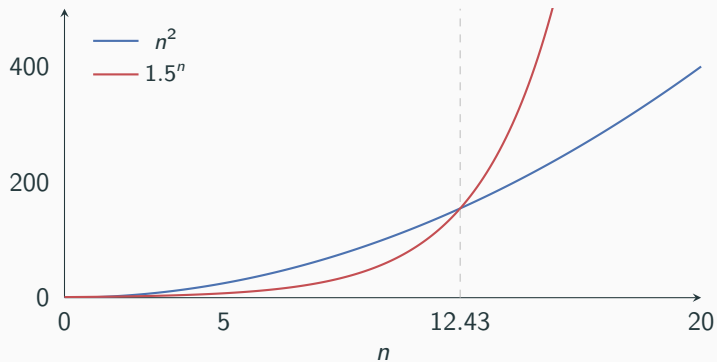
Example:  $n^2 = O(1.5^n)$

Take  $c = 1$  and  $n_0 = 12.43$ .



## Example: $n^2 = O(1.5^n)$

Take  $c = 1$  and  $n_0 = 12.43$ .



From  $n = 12.43$  onward,  $n^2 \leq 1.5^n$ .

# The Limit Approach

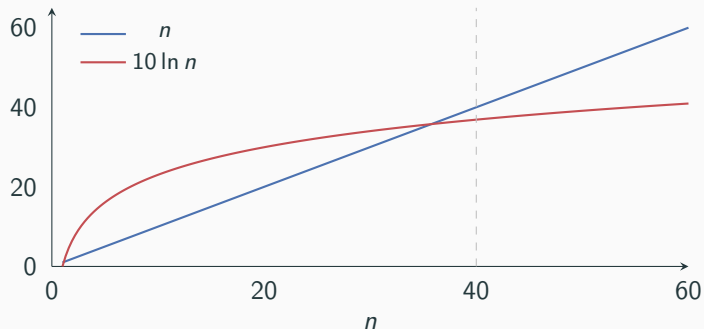
If  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \text{constant} < \infty$ , then  $f(n) = O(g(n))$ .

**Example:**

$$\lim_{n \rightarrow \infty} \frac{n^2/2 + n/2 - 1}{n^2} = \lim_{n \rightarrow \infty} \left( \frac{1}{2} + \frac{1}{2n} - \frac{1}{n^2} \right) = \frac{1}{2}.$$



Example:  $n + 10 \ln n = O(n)$



For  $n \geq 40$  we have  $10 \ln n \leq n$ , hence

$$n + 10 \ln n \leq n + n = 2n \quad (c = 2, n_0 = 40).$$

### L'Hôpital's rule

If  $\lim f'(n)/g'(n)$  exists, then  $\lim \frac{f(n)}{g(n)} = \lim \frac{f'(n)}{g'(n)}$ .

$$\lim_{n \rightarrow \infty} \frac{n + 10 \ln n}{n} = \lim_{n \rightarrow \infty} \frac{1 + 10/n}{1} = 1 < \infty.$$

1. Show that  $\log_a n = O(\log_b n)$  for constants  $a, b$ .

Hint: change-of-base formula.

2. Show that  $2\sqrt{n} + n^{1/3} \log_2 n = O(\sqrt{n})$ .

## Solution 1

Change of base:  $\log_a n = \frac{\log_b n}{\log_b a}$ .

Since  $\log_b a$  is a **constant**, take  $c = 1/\log_b a$  and  $n_0 = 1$ :

$$\log_a n = c \cdot \log_b n \leq c \cdot \log_b n.$$

The base of a logarithm never matters inside  $O(\cdot)$ .

## Solution 2

We need  $n^{1/3} \log_2 n$  to be swallowed by  $\sqrt{n} = n^{1/2}$ :

$$\frac{n^{1/3} \log_2 n}{n^{1/2}} = \frac{\log_2 n}{n^{1/6}} \xrightarrow{n \rightarrow \infty} 0,$$

since a polynomial beats a logarithm.

So  $n^{1/3} \log_2 n \leq \sqrt{n}$  for large  $n$ , giving

$$2\sqrt{n} + n^{1/3} \log_2 n \leq 3\sqrt{n} \quad (c = 3).$$

## Definition

$g(n) = \Omega(f(n))$  if  $f(n) = O(g(n))$ .

$$f(n) = O(g(n))$$

grows **no faster** than  $g$

$$f(n) = \Omega(g(n))$$

grows **no slower** than  $g$

## Definition

$f(n) = \Theta(g(n))$  if  $f(n) = O(g(n))$  and  $g(n) = O(f(n))$ .

Equivalently, via limits:

$$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty.$$

Neither function grows strictly faster than the other.

Example:  $2n^{100} + n^{99} = \Theta(n^{100})$

$$\lim_{n \rightarrow \infty} \frac{2n^{100} + n^{99}}{n^{100}} = 2 \quad \implies \quad 2n^{100} + n^{99} = O(n^{100}).$$

$$\lim_{n \rightarrow \infty} \frac{n^{100}}{2n^{100} + n^{99}} = \lim_{n \rightarrow \infty} \frac{1}{2 + 1/n} = \frac{1}{2} \quad \implies \quad n^{100} = O(2n^{100} + n^{99}).$$

Both directions hold, so the two functions have the **same growth**.



1. Show that  $n^{1.99} = O(n^2)$  but  $n^{1.99} \neq \Theta(n^2)$ .
2. Show that if  $n$  is a power of 2, then  $2 + 4 + 8 + \dots + n = \Theta(n)$ .

## Solution 1

**Upper bound.**  $\lim_{n \rightarrow \infty} \frac{n^{1.99}}{n^2} = \lim_{n \rightarrow \infty} n^{-0.01} = 0 < \infty$ , so  $n^{1.99} = O(n^2)$ .

**Not  $\Theta$ .** For  $\Theta$  we would also need  $n^2 = O(n^{1.99})$ , but

$$\lim_{n \rightarrow \infty} \frac{n^2}{n^{1.99}} = \lim_{n \rightarrow \infty} n^{0.01} = \infty.$$

No constant  $c$  can satisfy  $n^2 \leq c n^{1.99}$  for all large  $n$ . In fact  $n^{1.99} = o(n^2)$ .

## Solution 2

Write  $n = 2^m$ . The sum is a geometric series:

$$2 + 4 + \dots + 2^m = \left( \sum_{i=0}^m 2^i \right) - 1 = \frac{2^{m+1} - 1}{2 - 1} - 1 = 2 \cdot 2^m - 2 = 2n - 2.$$

Since  $n \leq 2n - 2 \leq 2n$  for  $n \geq 2$ , the sum is  $\Theta(n)$ .

A doubling sum is dominated by its **last** term.

# Little- $o$ and Little- $\omega$

## Definition

$f(n) = o(g(n))$  if for *any* constant  $c > 0$  there is  $n_0$  with  $f(n) < c \cdot g(n)$  for all  $n \geq n_0$ .

Equivalently  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ .

## Definition

$f(n) = \omega(g(n))$  if  $g(n) = o(f(n))$ . Equivalently  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$ .

Strictly slower and strictly faster growth.

**Claim:**  $\log_2 n = o(n^{0.1})$ .

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{n^{0.1}} = \frac{1}{\ln 2} \lim_{n \rightarrow \infty} \frac{\ln n}{n^{0.1}} = \frac{1}{\ln 2} \lim_{n \rightarrow \infty} \frac{1/n}{0.1 n^{-0.9}} = \frac{1}{\ln 2} \lim_{n \rightarrow \infty} \frac{10}{n^{0.1}} = 0.$$

**Claim:**  $3^n = \omega(2^n)$ .

$$\lim_{n \rightarrow \infty} \frac{3^n}{2^n} = \lim_{n \rightarrow \infty} (3/2)^n = \infty.$$

# Recap

---

|                       |                                        |
|-----------------------|----------------------------------------|
| $f(n) = O(g(n))$      | growth <b>at most</b> $g$              |
| $f(n) = \Omega(g(n))$ | growth <b>at least</b> $g$             |
| $f(n) = \Theta(g(n))$ | growth the <b>same</b> as $g$          |
| $f(n) = o(g(n))$      | growth <b>strictly slower</b> than $g$ |
| $f(n) = \omega(g(n))$ | growth <b>strictly faster</b> than $g$ |

---



## Useful Facts

---

## Fact 1: Polynomials

A polynomial grows like its **most significant term**:

$$3n^3 - n^2 + 100n = \Theta(n^3).$$

A polynomial always beats a poly-logarithm:

$$n = \omega(\log^{999} n),$$

$$n^{0.1} = \omega(\log^{99} n),$$

$$n \log^2 n = o(n^{1.01}).$$

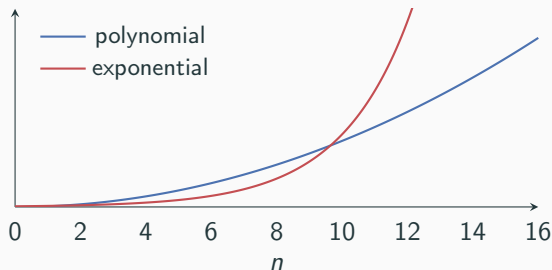


## Fact 2: Exponentials

An exponential (base  $> 1$ ) always beats a polynomial:

$$n^{99} = o(2^n),$$

$$n^{10} = o(1.01^{2^n}).$$



## Fact 3: Power Sums

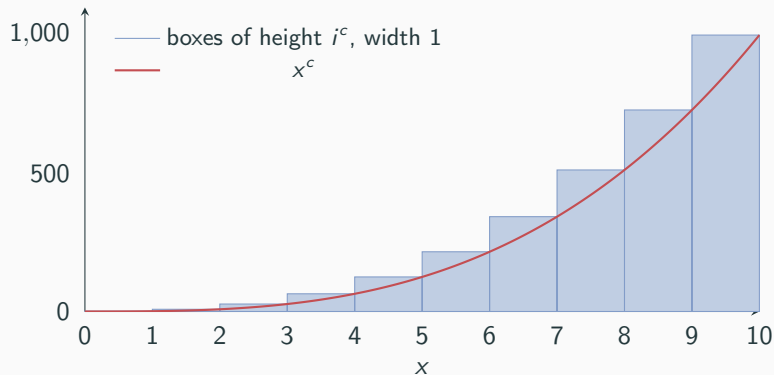
For any constant  $c$ :  $1^c + 2^c + \dots + n^c = \Theta(n^{c+1})$ .

**Upper bound.** Every term is at most  $n^c$ :

$$1^c + 2^c + \dots + n^c \leq \underbrace{n^c + n^c + \dots + n^c}_{n \text{ times}} = n^{c+1}.$$

So the sum is  $O(n^{c+1})$ . The lower bound needs more work.

# Approximation by Integrals



Box area =  $1^c + 2^c + \dots + n^c \geq$  area under the curve.

## Fact 3: Lower Bound

Each box at position  $i$  has width 1 and height  $i^c$ , so the boxes cover the curve:

$$1^c + 2^c + \dots + n^c \geq \int_0^n x^c dx = \frac{n^{c+1}}{c+1}.$$

Hence the sum is  $\Omega(n^{c+1})$ , and combined with the upper bound,

$$1^c + 2^c + \dots + n^c = \Theta(n^{c+1}).$$

$$\text{E.g. } 1^2 + 2^2 + \dots + n^2 = \Theta(n^3).$$

### Fact 4

For any constant  $c$ ,  $c = O(1)$ .

### Fact 5

If  $a < b$ , then  $a^n = o(b^n)$ . E.g.  $2^n = o(3^n)$ .

# A Common Pitfall

The following proof is **wrong**:

$$1 + 2 + 3 + \dots + n = O(1) + O(1) + \dots + O(1) = O(n).$$

But we know  $1 + 2 + \dots + n = \frac{n(n+1)}{2} = \Theta(n^2)$ .

## What went wrong

For a *fixed*  $i$ , yes,  $i = O(1)$ .

Inside  $\sum_{i=1}^n i$ , the value  $i$  ranges up to  $n$ , so it is **not** a constant.

Be careful with sums whose number of terms is not fixed.